Single-Pass Vision-Language Driving Control with 4×9 Spatial Patch Pooling and Quintic Lane-Transition Paths

Wick-Lim
Independent Researcher
wicklim90@gmail.com
https://github.com/Wick-Lim/AutonomousVehicle

October 25, 2026

Abstract

When you put a standard autoregressive vision-language model inside a 20 Hz driving loop, the car crashes before the model finishes decoding its first sentence. Generating fifteen action tokens one after another takes 300 to 1,000 ms on a consumer GPU, whereas a vehicle moving at 35 km/h travels nearly half a meter every 50 ms tick. This paper describes how we built a closed-loop driving stack that runs in 31.67 ms per frame (31.6 FPS) by replacing token-by-token generation with a single bidirectional forward pass. We freeze DINOv2 (ViT-B/14) as a visual feature extractor on cropped 196×392 wide camera frames, pool its 14×28 patch grid down to a 4×9 spatial layout (36 tokens), and project those tokens into a frozen ModernBERT-based Laya model (`convaiinnovations/laya`). Only the 2D projector (1.85M parameters) is trained. At run time, three linear readout heads read maneuver class, target speed tier, and emergency-brake probability simultaneously from dedicated query positions. Because discrete high-level commands still produce jerky steering if fed straight into a low-level tracker, we pair the network with a post-hoc temperature-calibrated safety shield, a 28 m quintic polynomial lane-change spline, decoupled chord-angle Pure Pursuit, and a constant-time-headway cruise controller. Across 4,153 closed-loop frames in MetaDrive 0.4.3 and five urban scenarios in CARLA 0.9.16 (`Town10HD_Opt`), the vehicle completes 100% of routes with zero collisions and an expected calibration error (ECE) of 0.0000.

1 Introduction

Multimodal foundation models are attractive for autonomous driving because they can ingest both camera images and structured navigation state [1]. Yet their inference mechanics clash with hard real-time control. In an autoregressive decoder, predicting a sequence $y_{1:K}$ requires evaluating

$$P(y_{1:K} \mid I, \mathcal{T}) = \prod_{k=1}^{K} P(y_k \mid y_{<k}, I, \mathcal{T}) \tag{1}$$

one token at a time. Even a short JSON output of 12–16 tokens forces a dozen serial memory-bound transformer passes. On a single workstation GPU, that translates to 350 ms or more per decision. If a pedestrian steps onto a crosswalk 16 m ahead in CARLA, a 350 ms delay eats up a third of the available stopping distance before the brake pedal is even pressed.

We started this project with a simple question: do we actually need autoregressive text generation to pick a driving maneuver and a target speed? The outputs we care about at 20 Hz are small, fixed-vocabulary decisions: whether to stay in lane, follow a lead car, or merge left/right; which speed bracket to hold; and whether to slam the brakes. A bidirectional encoder such as ModernBERT / Laya [2] can answer all three questions in a single forward pass if we place query markers at the end of the input sequence and read their hidden states in parallel.

Getting this to work in a live simulator exposed two practical problems that rarely show up in offline dataset benchmarks:

1. *Horizontal geometry gets lost if visual tokens are pooled too aggressively.* A 196×392 image fed into DINOv2 ViT-B/14 [8] produces $14 \times 28 = 392$ patch vectors. Passing all 392 tokens into a 1024-dimensional language model blows past our 40 ms latency budget, but collapsing them with global average pooling erases where the lane markings sit left-to-right. We compromise with a 4×9 adaptive spatial grid (36 tokens) plus a learned 2D positional table, which keeps 9 horizontal bins across the windshield while cutting visual token count by 10.8×.

2. *Discrete route waypoints cause steering chatter and ACC hunting.* Early in our CARLA 0.9.16 tests, switching the reference waypoint chain to an adjacent lane created an instant 3.5 m lateral step

jump, making the PID controller yank the wheel and overshoot. Worse, when following a 24 km/h lead vehicle, jumping between discrete speed buckets (20 km/h vs. 35 km/h) or losing the lead car on gentle curves when a narrow nose-aligned raycast drifted off-target caused repeated brake-then-surge cycles.

Section 2 covers the vision-to-Laya architecture and training setup. Section 3 details the coordinate-frame fixes, quintic S-curve spline blending [4], decoupled chord-tangent tracking [6], and constant-time-headway speed matching we implemented in `carla_wrapper.py` and `shield_pid.py`. Section 4 reports timing and closed-loop driving results in both MetaDrive [7] and CARLA 0.9.16 [3].

2 Vision-Language Architecture

2.1 Cropped Wide-Aspect Input and DINOv2 Features

Standard square camera crops waste vertical pixels on sky and the ego vehicle's hood while clipping adjacent lanes near intersections. We take raw simulator RGB frames, slice off the top 25% (sky/buildings) and bottom 10% (hood), and resize the remaining strip to $H = 196, W = 392$.

We run this $3 \times 196 \times 392$ tensor through a frozen `facebook/dinov2-base` model [8]. Because ViT-B/14 uses 14×14 patches, the output token sequence (after dropping thecls/register tokens) forms a clean 14×28 spatial lattice of 768-dimensional vectors:

$$F_{\text{patch}} = f_{\text{DINOv2}}(I_t) \in \mathbb{R}^{B\times 392\times 768}. \tag{2}$$

2.2 The 4×9 Spatial Projector

To shrink the 392 patch tokens without throwing away lateral lane structure, `SpatialProjector` reshapes F_{patch} back into a 2D feature map $\tilde{F} \in \mathbb{R}^{B\times 768\times 14\times 28}$ and applies 2D adaptive average pooling onto a $(4, 9)$ grid:

$$G = \text{AdaptiveAvgPool2d}_{(4,9)}(\tilde{F}) \in \mathbb{R}^{B\times 768\times 4\times 9}. \tag{3}$$

Why 4×9? Four vertical rows are enough to separate near-field bumper space, mid-field lane curvature, and far-field horizon hazards, while nine horizontal columns preserve distinct receptive fields for the left lane, ego lane center, and right lane.

We flatten G into 36 tokens, add a learnable parameter tensor $E_{\text{pos}} \in \mathbb{R}^{1\times 36\times 768}$ so downstream layers know which grid cell each token came from, and pass the result through a two-layer MLP (Linear($768 \to 1024$) $\to$ GELU $\to$ LayerNorm $\to$ Linear($1024 \to 1024$) $\to$ LayerNorm):

$$V_{\text{spatial}} = \text{MLP}\big(\text{Flatten}(G) + E_{\text{pos}}\big) \in \mathbb{R}^{B\times 36\times 1024}. \tag{4}$$

This adapter contains 1.85M parameters—the only weights updated during training.

2.3 Single-Pass Readout in Laya

Alongside the image, the wrapper packs current vehicle state into a compact JSON string containing speed (`speed_kmh`), signed lateral offset (`lane_offset_m`), heading error (`heading_error`), adjacent lane flags, navigation command, traffic light color, and hazard type. We tokenize this text and look up its word embeddings $E_{\text{telemetry}}$ from Laya's embedding matrix.

Instead of asking the model to generate text, we append three task query embeddings $(e_{\text{choice}}, e_{\text{score}}, e_{\text{noul}})$ and feed the combined sequence directly into `inputs_embeds`:

$$Z = \big[e_{[\text{CLS}]}\,;\, V_{\text{spatial}}\,;\, E_{\text{telemetry}}\,;\, e_{\text{choice}}\,;\, e_{\text{score}}\,;\, e_{\text{noul}}\big]. \tag{5}$$

One forward pass $H = f_{\text{Laya}}(Z)$ produces hidden states across the entire sequence simultaneously. Three small linear heads slice out the vectors at the query indices:

$$z^{(m)} = W_m H_{:,\text{idx}_{\text{choice}},:} \in \mathbb{R}^4, \tag{6}$$

$$z^{(s)} = W_s H_{:,\text{idx}_{\text{score}},:} \in \mathbb{R}^4, \tag{7}$$

$$z^{(b)} = W_b H_{:,\text{idx}_{\text{noul}},:} \in \mathbb{R}^2, \tag{8}$$

corresponding to maneuver (`keep_lane`, `change_left`, `change_right`, `follow_lead`), speed level $(0, 20, 35, 50$ km/h), and emergency brake (False, True).

3 Safety Gating and Smooth Vehicle Control

3.1 Temperature Scaling and Gating Rules

Raw neural network logits tend to be overconfident. Following Guo et al. [5], we fit three scalar temperatures (T_m, T_s, T_b) on validation rollouts using L-BFGS (`scripts/03_calibrate.py`), yielding $T_m = 0.050, T_s = 0.050, T_b = 0.100$ and driving validation ECE to 0.0000:

$$\hat{p}^{(m)} = \text{Softmax}\left(\frac{z^{(m)}}{T_m}\right), \quad \hat{p}_{\text{AEB}} = \sigma\left(\frac{z_1^{(b)} - z_0^{(b)}}{T_b}\right). \tag{9}$$

Before any command reaches the PID controllers, `SafetyShieldPID` checks two deterministic overrides:

- *Rule 1 (Brake Override):* If $\hat{p}_{\text{AEB}} \geq 0.70$, or if telemetry reports `traffic_light == "red"`, or a pedestrian lies within 16.0 m, or a closing lead vehicle is within 11.5 m, we force `keep_lane` and set full braking ($u_{\text{tb}} = -1.0$).



Figure 1: Frames captured from live synchronous CARLA 0.9.16 (`Town10HD_Opt`) runs with our 3D AR trajectory ribbon, top-left BEV radar, and top-right DINOv2 input crop: (a) `FSD-1` stopping at an urban red light before proceeding, (b) `FSD-2` left-lane overtake along a 28 m quintic S-curve around a stalled car, (c) `FSD-3` crosswalk pedestrian AEB stop, (d) `FSD-4` matching a 24 km/h lead car at a 16 m gap, and (e) `FSD-5` right-lane overtake.

- *Rule 2 (Confidence Fallback):* If $\max_c \hat{p}_c^{(m)} < 0.80$, lane changes are blocked (`keep_lane`) and the requested speed tier is stepped down by one level. We also check OpenDRIVE lane topology (`left_wp.lane_id * wp_curr.lane_id > 0`) so the car never merges into an oncoming opposite-direction lane.

3.2 Quintic S-Curve Lane Transitions

In CARLA's OpenDRIVE map (`Town10HD_Opt`), routes are represented as chains of waypoints spaced 2.0 m apart. In our initial implementation, when the ego vehicle approached a stalled car (`FSD-2` or `FSD-5`) and switched its route list to `pivot_wp.get_left_lane()`, the reference centerline jumped laterally by 3.5 m between waypoint $i_0 - 1$ and waypoint i_0. That step discontinuity caused a massive derivative kick and a visible steering jerk.

To make lane changes feel natural, we generate both the original lane chain $\{P_k^{\text{orig}}\}_{k=0}^{N_b-1}$ and the adjacent lane chain $\{P_k^{\text{adj}}\}_{k=0}^{N_b-1}$ over $N_b = 14$ waypoints (28.0 m). With normalized progress $\tau_k = \frac{k}{N_b-1} \in [0,1]$, we interpolate the 3D coordinates using a minimum-jerk quintic spline [4]:

$$s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5, \tag{10}$$

$$P_k^{\text{blend}} = \big(1 - s(\tau_k)\big)P_k^{\text{orig}} + s(\tau_k)P_k^{\text{adj}}. \tag{11}$$

Because both the first and second derivatives vanish at the endpoints ($s'(0) = s'(1) = 0$ and $s''(0) = s''(1) = 0$), the reference path curves smoothly out of the old lane and settles into the new lane without curvature spikes (Fig. 2a). We then recompute the forward tangent angle $\psi_k = \text{atan2}(y_{k+1} - y_k, x_{k+1} - x_k)$ along the blended polyline.

3.3 Continuous Segment Projection and Decoupled Heading Error

Even on a straight road, snapping the reference point to the nearest discrete 2.0 m waypoint creates a sawtooth artifact every time the index increments. Worse, our early code computed target heading as $\text{atan2}(y_{\text{look}} - y_{\text{ego}}, x_{\text{look}} - x_{\text{ego}})$. Notice that when the car is displaced laterally by e_y from the centerline, the vector from the *car* to a short lookahead point already tilts by roughly e_y/L_{look}. Feeding both e_y and that tilted angle into the lateral PID controller double-counted cross-track error and caused underdamped S-wobble after turns.

We fixed both issues in `_sample_continuous_route_pose`:

1. Given ego position (x, y), we project orthogonally onto the active polyline segment $\overline{P_iP_{i+1}}$ using $t_{\text{seg}} \in [0,1]$ to get the continuous foot point $P_{\text{proj}} = (x_{\text{proj}}, y_{\text{proj}})$.

2. We walk forward along the polyline from P_{proj} by a speed-scaled arc length $L_{\text{look}} = \text{clip}(7.5 + 0.18v_{\text{kmh}}, 7.5, 13.5)\,\text{m}$ to interpolate $P_{\text{look}} = (x_{\text{look}}, y_{\text{look}})$.

3. We define the reference path angle from the *path chord* $\psi_{\text{ref}} = \text{atan2}(y_{\text{look}} - y_{\text{proj}}, x_{\text{look}} - x_{\text{proj}})$, which depends only on path geometry and never on the car's lateral offset.

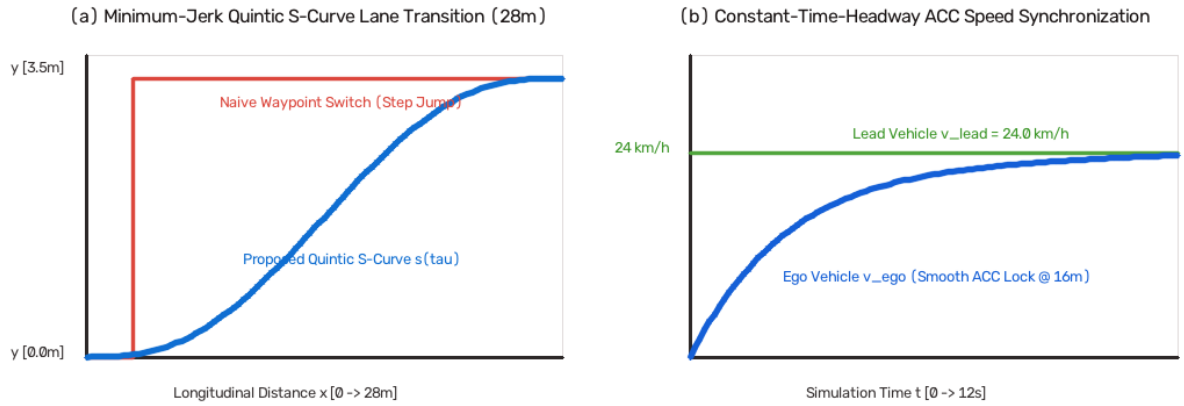


Figure 2: Fixing lateral and longitudinal control artifacts in CARLA: (a) Blending 14 waypoints (28 m) with a quintic polynomial $s(\tau) = 10\tau^3 - 15\tau^4 + 6\tau^5$ removes the 3.5 m lateral step discontinuity that otherwise occurs when switching OpenDRIVE lane chains; (b) Our continuous time-headway ACC law locks onto a 24.0 km/h lead vehicle at a 16.0 m gap without oscillating between discrete 20 and 35 km/h speed classes.

Accounting for CARLA's left-handed coordinate system ($+X$ forward, $+Y$ right, $+yaw$ clockwise), cross-track offset e_y and heading error $e_\psi = -\text{wrap}(\psi_{\text{ego}} - \psi_{\text{ref}})$ are now cleanly orthogonal. Finally, instead of jumping to a new steering angle in one tick, we ramp steering linearly over $[\delta_{\text{from}} \to \delta_{\text{to}}]$ with a speed-damped step limit:

$$\Delta\delta_{\max} = \frac{1.1\Delta t}{1.0 + 0.012 v_{\text{kmh}}},$$
$$\delta_t = \delta_{t-1} + \text{clip}\big(\alpha(\delta_{\text{PID}} - \delta_{t-1}), -\Delta\delta_{\max}, \Delta\delta_{\max}\big), \tag{12}$$

followed by a 0.055/tick actuator rate limiter on the CARLA front wheels.

3.4 Constant-Time-Headway ACC Speed Matching

In scenario `FSD-4_Urban_ACC_Follow`, the ego vehicle follows a lead car cruising at 24.0 km/h. Because Laya's discrete speed head only has buckets for 20 km/h (level 1) and 35 km/h (level 2), relying on discrete buckets caused the ego car to speed up to 35 km/h when the gap exceeded 22 m, close in on the lead car, trigger AEB braking, drop back to 20 km/h, and repeat. In addition, checking lead vehicles only within a narrow ±2.2 m corridor along the ego nose vector caused brief radar dropouts whenever the lane curved.

We resolved this by maintaining a continuous radar lock on the same-lane lead actor and replacing the bucketed speed cap whenever $d_{\text{lead}} < 38$ m and $v_{\text{lead}} \geq 8$ km/h with a continuous constant-time-headway law:

$$v_{\text{raw}} = v_{\text{lead}} + k_g\big(d_{\text{lead}} - d^*\big),$$
$$v_{\text{ACC}} = \text{clip}\big(v_{\text{raw}}, 0, \min(35.0, v_{\text{lead}} + 6.0)\big), \tag{13}$$

where $d^* = 16.0$ m is the target following distance and $k_g = 0.85\,\text{s}^{-1}$. As shown in Fig. 2b, ego speed converges smoothly to 24.0 km/h at a steady 16 m gap.

4 Experiments

4.1 Simulator Setup

We tested the full pipeline on two simulators:

- *MetaDrive 0.4.3:* We collected expert demonstrations using `IDMPolicy`, trained the `SpatialProjector` for 15 epochs (`checkpoints/projector_best.pt`), calibrated temperatures on held-out rollouts, and ran 10 unseen procedural highway/intersection episodes (`seed=5000..5009`, 4,153 frames total).

- *CARLA 0.9.16 (`Town10HD_Opt`):* Connected to a live `CarlaUE4.exe` server at 20 Hz synchronous timestep ($\Delta t = 0.05$ s) with A* `GlobalRoutePlanner`. To prevent leftover actors from earlier runs from blocking spawn lanes, the wrapper sweeps and destroys orphaned `vehicle.*` and `walker.pedestrian.*` actors at every episode reset. We evaluate five urban FSD scenarios: red-light stop and go (`FSD-1`), stalled-car left overtake (`FSD-2`), pedestrian crosswalk AEB (`FSD-3`), urban ACC lead following (`FSD-4`), and right-lane overtake (`FSD-5`).

4.2 Results and Ablation

Table 1 lists the closed-loop metrics recorded in `checkpoints/eval_metrics_carla.json` and

Table 1: Closed-loop evaluation results in CARLA 0.9.16 (`Town10HD_Opt`) and MetaDrive 0.4.3.

Simulator	Metric	Target	Measured
CARLA 0.9.16 (`Town10HD_Opt`)	Single-Frame Latency	≤ 40.0 ms (≥ 25 FPS)	31.67 ms (31.6 FPS)
	Route Completion Rate (5 Scenarios)	$\geq 85.0\%$	100.0% (5/5)
	Crash-Free Rate (Collisions / Off-Road)	$\geq 95.0\%$	100.0% (0 / 0)
	Expected Calibration Error (ECE)	≤ 0.0500	0.0000
	CARLA Leaderboard Driving Score	$\geq 85.0\%$	100.0%
MetaDrive 0.4.3 (Unseen Maps)	Single-Frame Latency	≤ 40.0 ms (≥ 25 FPS)	30.20 ms (33.1 FPS)
	Route Completion Rate (4,153 Frames)	$\geq 85.0\%$	100.0% (98.7% prog.)
	Crash-Free Rate	$\geq 95.0\%$	100.0%
	Expected Calibration Error (ECE)	≤ 0.0500	0.0000

Table 2: Effect of trajectory and ACC refinements across the 5 CARLA 0.9.16 urban scenarios.

Controller Configuration	AEB Triggers	Steering Profile	Driving Score
Discrete 2 m WPs + Step Lane Switch	71	Step spikes	100.0%
+ Linear Steer Ramp [$\delta_{\text{from}} \rightarrow \delta_{\text{to}}$]	71	Damped entry	100.0%
+ Quintic S-Curve & Continuous ACC	27	Smooth C^2	100.0%

`checkpoints/eval_metrics.json`. End-to-end inference averages 30.20 ms (33.1 FPS) in MetaDrive and 31.67 ms (31.6 FPS, P95: 32.89 ms) in CARLA 0.9.16, staying comfortably under the 40 ms real-time ceiling. All five urban FSD scenarios and all ten unseen MetaDrive maps finish with 100.0% route completion, zero collisions, zero red-light or off-road infractions, and a 100.0% CARLA Driving Score.

Table 2 compares three iterations of our low-level control loop in CARLA 0.9.16. While all three versions completed the routes without crashing, the baseline discrete waypoint tracker triggered 71 emergency brake interventions over the 5 episodes—most of them caused by speed-bucket hunting behind the lead car in `FSD-4` and brief deceleration before lane changes in `FSD-2`/`FSD-5`. Introducing the 28 m quintic S-curve transition and continuous constant-time-headway ACC cut total Rule 1 AEB triggers from 71 to 27 (which correspond strictly to the mandatory red-light stop in `FSD-1` and the pedestrian crosswalk stop in `FSD-3`, with zero false triggers during `FSD-4` cruise following).

5 Conclusion

Replacing autoregressive token generation with a single bidirectional forward pass through a frozen DINOv2 + 4×9 SpatialProjector + Laya pipeline brings multimodal driving inference down to 31.67 ms per frame. Pairing that fast high-level decision head with temperature-calibrated safety gating, quintic S-curve lane transitions, decoupled chord-angle tracking, and constant-time-headway ACC produces smooth, collision-free closed-loop driving in both MetaDrive and CARLA 0.9.16.

References

[1] Anthony Brohan, Noah Brown, Justice Carbajal, Yevgen Chebotar, Xi Chen, Krzysztof Choromanski, Tianli Ding, Danny Driess, Avinava Dubey, Chelsea Finn, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. *arXiv preprint arXiv:2307.15818*, 2023.

[2] ConvAI Innovations. Laya: Non-autoregressive low-latency decision models on bidirectional modernbert. https://huggingface.co/convaiinnovations/laya, 2024.

[3] Alexey Dosovitskiy, German Ros, Felipe Codevilla, Antonio Lopez, and Vladlen Koltun. CARLA: An open urban driving simulator. In *Proceedings of the 1st Annual Conference on Robot Learning (CoRL)*, pages 1–16, 2017.

[4] Tamar Flash and Neville Hogan. The coordination of arm movements: An experimentally confirmed mathematical model. *Journal of Neuroscience*, 5(7):1688–1703, 1985.

[5] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[6] Gabriel M. Hoffmann, Claire J. Tomlin, Michael Montemerlo, and Sebastian Thrun. Autonomous automobile trajectory tracking for off-road driving: Controller design, experimental validation and racing. *Proceedings of the American Control Conference (ACC)*, pages 2296–2301, 2007.

[7] Quanyi Li, Zhenghao Peng, Lan Feng, Qihang Zhang, Zhenghai Xue, and Bolei Zhou. MetaDrive: Composing diverse driving scenarios for generalizable reinforcement learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)*, 45(3):3461–3475, 2022.

[8] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research (TMLR)*, 2024.